

SESSIONAL EXAM HANDBOOK

Web Technology

HTML & CSS

XML & DTD

Client–Server

Internet Basics

Complete Revision Edition

Table of Contents

1	What Are Internet Basics?	Definitions & Concepts
2	What Is Web Technology?	Overview
3	What Do You Mean by HTML Tag?	Definition & Types
4	Explain Anchor Tag	Syntax & Attributes
5	What Is a Web Project?	Lifecycle
6	Uses of CSS	Benefits
7	Difference Between SAX and DOM	Comparison Table
8	HTML vs XML Difference	Comparison Table
9	Table Tag with Example	Full Code
10	Student Registration Form	Full Code
11	Why JavaScript Uses DOM	Explanation
12	Font Tag with Attributes	Table & Example
13	5 Milestones of Internet History	Timeline
14	Functions of HTTP, HTTPS, FTP, SMTP	Comparison Table
15	Client–Server Architecture Cycle	Diagram
16	Explain DTD	Definition & Example
17	CSS Code to Design a Web Page	Full Code
18	Form Tag with Example	Full Code

The **Internet** is a massive, global network of interconnected computer networks that communicate using a common set of rules called **protocols** (chiefly TCP/IP — Transmission Control Protocol / Internet Protocol). It allows billions of devices worldwide to exchange data, regardless of their hardware, operating system, or physical location.

Core Building Blocks of the Internet

Concept	Explanation
IP Address	A unique numerical label (e.g., 192.168.1.1 or IPv6) assigned to every device connected to a network, used to identify and locate it.
Protocol	A set of agreed-upon rules that govern how data is formatted, transmitted, and received (e.g., TCP/IP, HTTP, FTP).
ISP (Internet Service Provider)	A company (e.g., Jio, Airtel, BSNL) that provides individuals and organizations access to the Internet.
Domain Name	A human-readable address (e.g., www.example.com) that maps to a numeric IP address via DNS.
DNS (Domain Name System)	The "phonebook of the Internet" — translates domain names into IP addresses.
Router	A networking device that forwards data packets between different networks.
Bandwidth	The maximum rate of data transfer across a network path, usually measured in Mbps/Gbps.
Packet	A small unit into which data is broken down for transmission across a network, reassembled at the destination.

Internet vs World Wide Web (WWW)

A very common point of confusion — the **Internet** is the physical/logical network infrastructure (cables, routers, protocols) that connects computers globally, while the **World Wide Web** is just one *service* that runs on top of the Internet — a system of interlinked hypertext documents (web pages) accessed via browsers using HTTP/HTTPS. Email, FTP, and online gaming are other services that also run over the Internet, separate from the Web.

Key idea: Internet = the network (the "roads"). WWW = one application running on that network (one kind of "traffic" using those roads, alongside email, FTP, etc.).

How the Internet Works (Simplified Flow)

Your Device

↓ sends request via ISP

Router / ISP Network

↓ routed across backbone networks using IP addresses

Destination Server

↓ response sent back as data packets

Back to Your Device

Exam tip: Be ready to define IP Address, DNS, ISP, and Protocol individually, plus clearly distinguish Internet from WWW — this exact distinction is one of the most frequently asked one-mark/short questions.

Web Technology refers to the collection of tools, languages, protocols, and standards used to design, build, and interact with websites and web applications that run on the Internet. It encompasses everything from the visual layout of a page to how data moves between a browser and a server.

Broad Categories

Category	Purpose	Examples
Client-Side (Front-End)	Runs in the user's browser; controls structure, style, and interactivity of what the user sees	HTML, CSS, JavaScript
Server-Side (Back-End)	Runs on the web server; handles logic, database interaction, and generates dynamic content	PHP, Node.js, Python (Django/Flask), Java (Servlets/JSP), ASP.NET
Database Technologies	Store and manage persistent data used by web applications	MySQL, MongoDB, PostgreSQL
Markup & Styling Languages	Define structure and presentation of web content	HTML, XML, CSS
Web Protocols	Define rules for communication between client and server	HTTP, HTTPS, FTP, SMTP
Web Servers	Software that hosts websites and serves requested pages to clients	Apache, Nginx, IIS

Static vs Dynamic Web Technology

Aspect	Static Web Pages	Dynamic Web Pages
Content	Fixed — same content shown to every visitor	Changes based on user input, database, or time
Technology Used	Plain HTML/CSS	Server-side scripting + database (PHP, Node.js, etc.)
Example	A simple "About Us" page	A personalized social media feed

Model answer: "Web technology is the set of tools and standards used to build and run websites, broadly divided into client-side technologies (HTML, CSS, JavaScript) that control what the user sees, and server-side technologies (PHP, Node.js, databases) that handle logic and data — all communicating over web protocols like HTTP."

An **HTML tag** is a keyword enclosed in angle brackets (`< >`) that tells the browser how to structure and display content on a web page. Tags are the fundamental building blocks of every HTML document — they mark the beginning and end of an HTML "element."

Anatomy of a Tag

```
<p class="intro">This is a paragraph.</p>
```

Part	Meaning
Opening Tag <code><p></code>	Marks the start of the element
Attribute <code>class="intro"</code>	Provides additional information/configuration for the tag
Content	The actual text/data displayed to the user
Closing Tag <code></p></code>	Marks the end of the element (note the forward slash)

Types of HTML Tags

Type	Description	Example
Container (Paired) Tags	Have both an opening and a closing tag, and can wrap content inside	<code><p>...</p></code> , <code><div>...</div></code>
Empty (Void/Self-Closing) Tags	Do not have a closing tag or wrap content — they represent standalone elements	<code>
</code> , <code></code> , <code><hr></code> , <code><input></code>

Note: HTML tags themselves are *not case-sensitive* (`<P>` works the same as `<p>`), but lowercase is the universally recommended convention, and is mandatory in stricter markup like XHTML.

Model answer: "An HTML tag is a markup keyword surrounded by angle brackets that instructs the browser on how to interpret and render a piece of content. Tags may be container tags (requiring both opening and closing tags, e.g. `<p></p>`) or empty/void tags that stand alone (e.g. `<br>`), and can carry attributes that customize their behavior or appearance."

The **anchor tag** `<a>` is used to create **hyperlinks** — clickable links that navigate the user to another web page, a different section of the same page, an email address, a file, or an external resource.

Basic Syntax

```
<a href="https://www.example.com">Visit Example</a>
```

Important Attributes of the Anchor Tag

Attribute	Purpose	Example
<code>href</code>	Specifies the destination URL of the link (Hypertext REference)	<code>href="page2.html"</code>
<code>target</code>	Specifies where to open the linked document	<code>target="_blank"</code> (new tab)
<code>title</code>	Provides a tooltip shown on hover	<code>title="Go to homepage"</code>
<code>rel</code>	Defines the relationship between current and linked document (often used with security like <code>noopener</code>)	<code>rel="nofollow"</code>
<code>download</code>	Indicates the linked resource should be downloaded rather than navigated to	<code>download="file.pdf"</code>
<code>id</code> / <code>name</code>	Used to create in-page bookmarks that other anchors can jump to	<code>id="section2"</code>

Common Values for `target`

Value	Behavior
<code>_self</code>	Opens in the same tab/frame (default)
<code>_blank</code>	Opens in a new browser tab/window
<code>_parent</code>	Opens in the parent frame
<code>_top</code>	Opens in the full body of the window, breaking out of any frames

Worked Examples

```
<!-- External link opening in a new tab -->
<a href="https://www.google.com" target="_blank">Google</a>

<!-- Email link -->
<a href="mailto:someone@example.com">Send Email</a>

<!-- Internal page anchor (bookmark link) -->
<a href="#section2">Jump to Section 2</a>
```

...

```
<h2 id="section2">Section 2</h2>
```

Exam tip: Be ready to write at least 3 variants from memory: a normal link, an email link (`mailto:`), and an in-page bookmark link (`href="#id"`).

A **web project** is a planned, structured effort to design, develop, test, and deploy a website or web application to fulfill a specific purpose — such as an e-commerce store, a college portal, or a blog. It typically involves multiple stages, technologies, and often a team of people with different roles (designers, developers, testers).

Typical Phases of a Web Project (Web Development Life Cycle)

Phase	Description
1. Requirement Gathering / Planning	Understand the client's goals, target audience, and required features
2. Design	Create wireframes, mockups, and decide the visual look-and-feel (UI/UX)
3. Development (Coding)	Build the front-end (HTML/CSS/JS) and back-end (server logic, database) of the site
4. Testing	Check for bugs, broken links, cross-browser compatibility, and performance issues
5. Deployment	Publish the website onto a live web server, making it accessible to the public
6. Maintenance	Ongoing updates, bug fixes, security patches, and content updates after launch

Common Components of a Web Project

- **Front-end files** — HTML pages, CSS stylesheets, JavaScript scripts, images
- **Back-end logic** — Server-side scripts handling business logic (PHP, Node.js, Python, etc.)
- **Database** — Stores persistent data (user accounts, products, articles)
- **Configuration files** — Server settings, environment variables
- **Documentation** — Project reports, user manuals, technical specifications

Model answer: "A web project is an organized undertaking to plan, design, build, test, and deploy a website or web application. It typically progresses through requirement gathering, design, development, testing, deployment, and maintenance phases, and combines front-end components (HTML/CSS/JS), back-end logic, and a database to deliver a functional product to end users."

CSS (Cascading Style Sheets) is a stylesheet language used to describe the visual presentation of HTML documents — controlling layout, colors, fonts, spacing, and responsiveness, separately from the content structure defined by HTML.

Key Uses / Benefits of CSS

#	Use	Explanation
1	Separation of Content and Presentation	Keeps HTML focused on structure while CSS handles appearance, making code cleaner and easier to maintain
2	Consistent Styling Across Pages	A single CSS file can style multiple HTML pages uniformly, ensuring visual consistency across a whole site
3	Improved Page Load Speed	External CSS files are cached by the browser, reducing repeated downloads and speeding up subsequent page loads
4	Responsive Web Design	Media queries allow layouts to adapt automatically to different screen sizes (mobile, tablet, desktop)
5	Easier Maintenance	Updating a single CSS rule can instantly change the look of every page that references it
6	Enhanced User Experience	Enables animations, transitions, hover effects, and visually appealing layouts
7	Accessibility	Proper use of CSS (e.g., font sizing, color contrast) improves readability for all users, including those with visual impairments
8	Reduced File Size / Bandwidth	Combining styling rules externally avoids repeating inline style code throughout HTML, reducing overall page size

Three Ways to Apply CSS

Type	Description	Example
Inline CSS	Applied directly within an HTML tag using the <code>style</code> attribute	<code><p style="color:red;"></code>
Internal (Embedded) CSS	Defined within a <code><style></code> block inside the <code><head></code> of the HTML document	<code><style>p{color:red;}</style></code>
External CSS	Defined in a separate <code>.css</code> file, linked via the <code><link></code> tag	<code><link rel="stylesheet" href="style.css"></code>

Exam tip: If asked to "explain uses of CSS," structure your answer as: separation of concerns → consistency → maintainability → responsiveness → performance → accessibility. Mentioning all three ways to apply CSS earns bonus completeness marks.

Both **SAX** and **DOM** are APIs (parsing models) used to read and process XML documents programmatically, but they take fundamentally different approaches.

SAX — Simple API for XML

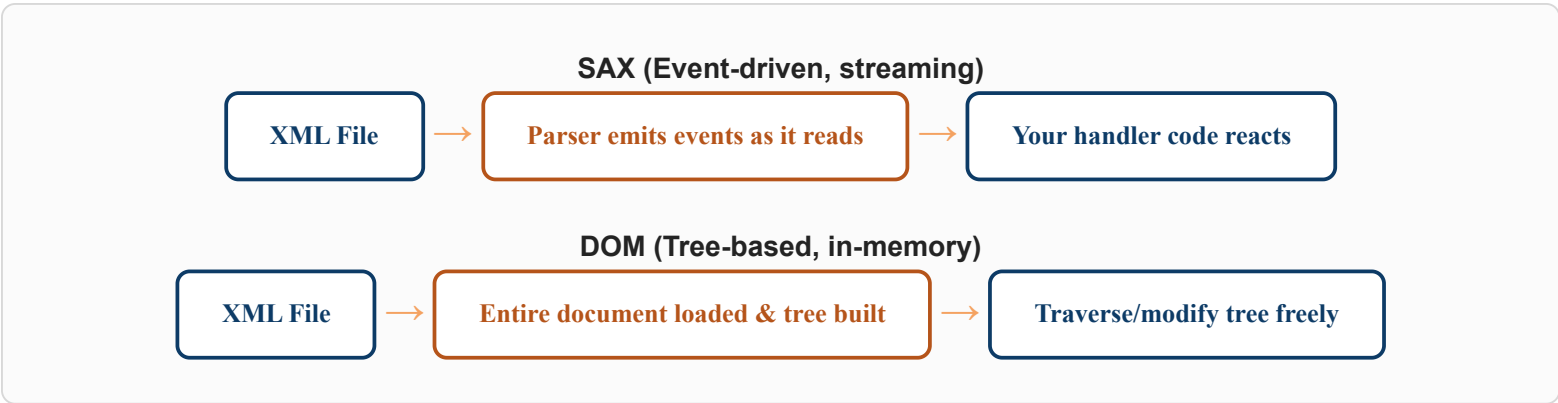
SAX is an **event-driven, sequential** parser. It reads the XML document from start to end, triggering "events" (like *start element, end element, character data*) as it encounters each piece of the document — without loading the entire document into memory.

DOM — Document Object Model

DOM is a **tree-based** parser. It reads the entire XML document and builds an in-memory tree representation of it, where every element, attribute, and piece of text becomes a node. This tree can then be freely traversed, modified, and navigated in any direction.

Comparison Table

Feature	SAX	DOM
Full form	Simple API for XML	Document Object Model
Parsing approach	Event-driven, sequential (streaming)	Tree-based, loads entire document
Memory usage	Low — processes data on the fly	High — entire document held in memory as a tree
Speed (large files)	Faster for large documents	Slower for large documents (due to full loading)
Navigation	Forward-only, cannot go back without re-parsing	Random access — can move freely in any direction
Modifying data	✗ Cannot modify the XML document (read-only)	✓ Can modify, add, or delete nodes in the tree
Best suited for	Large XML files where memory is limited, one-time read operations	Smaller XML files needing repeated access, editing, or complex navigation
API style	Callback/event-based	Object-oriented tree API



Model answer: "SAX is an event-driven, forward-only, read-only XML parsing model well suited to large files due to its low memory footprint, whereas DOM builds a complete, modifiable in-memory tree of the document that supports random access and editing, at the cost of higher memory usage — making DOM better for smaller documents requiring complex or repeated manipulation."

HTML (HyperText Markup Language) is used to **display** data with predefined formatting tags, while **XML (eXtensible Markup Language)** is used to **store and transport** data using custom, user-defined tags. Both are markup languages, but they serve fundamentally different purposes.

Comparison Table

Feature	HTML	XML
Full form	HyperText Markup Language	eXtensible Markup Language
Purpose	Designed to <i>display</i> data and define page structure	Designed to <i>store and transport</i> data
Tags	Predefined, fixed set of tags (e.g., <code><p></code> , <code><table></code>)	User-defined, custom tags (e.g., <code><book></code> , <code><author></code>)
Case sensitivity	Not case-sensitive	Case-sensitive (<code><Book></code> $\neq$ <code><book></code>)
Closing tags	Some tags can be left unclosed (e.g., <code>
</code> , <code></code> in HTML4)	Every tag <i>must</i> be properly closed — strict syntax rules
Nesting	Improper nesting is often tolerated by browsers	Improper nesting is not allowed — must be strictly well-formed
Focus	Presentation/appearance of data	Structure and meaning of data (data description)
Error tolerance	Browsers are forgiving of minor errors	Strict — a single syntax error can make the whole document invalid
Extensibility	Not extensible — fixed vocabulary defined by the HTML standard	Fully extensible — developers define their own tag vocabulary
Dependency	Static — cannot carry application-specific data structures on its own	Dynamic — commonly used to exchange data between different systems (APIs, config files)

Side-by-Side Example

HTML

```
<table>
  <tr>
    <td>Java</td>
  </tr>
</table>
```

XML

```
<book>
  <title>Java</title>
</book>
```

Model answer: "HTML uses a fixed set of predefined tags purely to display and format content in a browser, and is fairly error-tolerant, while XML allows developers to define their own custom tags purely to structure and transport data, enforcing

strict, case-sensitive, well-formed syntax. In short: HTML is about presentation, XML is about data description."

The `<table>` tag is used in HTML to organize and display data in a grid of **rows and columns**. It is composed of several related tags that together define the table's structure.

Key Table-Related Tags

Tag	Purpose
<code><table></code>	Defines the overall table container
<code><tr></code>	Defines a table row (Table Row)
<code><th></code>	Defines a header cell (Table Header) — bold and centered by default
<code><td></code>	Defines a standard data cell (Table Data)
<code><caption></code>	Defines a title/caption for the table
<code><thead></code> , <code><tbody></code> , <code><tfoot></code>	Group the header, body, and footer sections of a large table respectively

Common Attributes

Attribute	Purpose
<code>border</code>	Sets the width of the table border (in pixels)
<code>colspan</code>	Makes a cell span across multiple columns
<code>rowspan</code>	Makes a cell span across multiple rows
<code>cellpadding</code>	Space between cell content and cell border
<code>cellspacing</code>	Space between adjacent cells

Full Worked Example

```
<table border="1" cellpadding="8" cellspacing="0">
  <caption>Student Marks</caption>
  <tr>
    <th>Roll No</th>
    <th>Name</th>
    <th>Marks</th>
  </tr>
  <tr>
    <td>1</td>
    <td>Asif</td>
    <td>95</td>
  </tr>
  <tr>
    <td colspan="2">Total Students</td>
    <td>1</td>
  </tr>
```

```
</tr>
</table>
```

Rendered Output Preview:

Roll No	Name	Marks
1	Asif	95
Total Students		1

Exam tip: Always demonstrate `colspan` or `rowspan` in your worked example — examiners specifically look for this to confirm you understand table merging, not just basic rows/columns.

A student registration form demonstrates multiple HTML form elements working together: text inputs, radio buttons, checkboxes, dropdowns, date pickers, and a submit button.

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration Form</title>
</head>
<body>
  <h2>Student Registration Form</h2>
  <form action="submit.php" method="post">

    <label>Full Name:</label>
    <input type="text" name="fullname" placeholder="Enter your name" required><br><br>

    <label>Roll Number:</label>
    <input type="text" name="rollno" required><br><br>

    <label>Date of Birth:</label>
    <input type="date" name="dob"><br><br>

    <label>Email:</label>
    <input type="email" name="email" placeholder="you@example.com"><br><br>

    <label>Password:</label>
    <input type="password" name="pwd"><br><br>

    <label>Gender:</label>
    <input type="radio" name="gender" value="male"> Male
    <input type="radio" name="gender" value="female"> Female<br><br>

    <label>Branch:</label>
    <select name="branch">
      <option value="cse">Computer Science</option>
      <option value="ece">Electronics</option>
      <option value="me">Mechanical</option>
    </select><br><br>

    <label>Hobbies:</label>
    <input type="checkbox" name="hobby" value="reading"> Reading
    <input type="checkbox" name="hobby" value="sports"> Sports<br><br>

    <label>Address:</label><br>
    <textarea name="address" rows="3" cols="30"></textarea><br><br>

    <input type="submit" value="Register">
    <input type="reset" value="Clear">

  </form>
</body>
</html>
```

Element	Purpose
<input type="text">	Single-line text entry (name, roll number)
<input type="date">	Date picker for date of birth
<input type="email">	Validates that entered text follows an email format
<input type="password">	Masks entered characters for security
<input type="radio">	Allows selecting exactly one option from a group (Gender)
<input type="checkbox">	Allows selecting multiple options (Hobbies)
<select><option>	Dropdown list for a single choice (Branch)
<textarea>	Multi-line text entry (Address)
<input type="submit">	Submits the form data to the server

The **DOM (Document Object Model)** is a programming interface that represents an HTML/XML document as a structured **tree of objects**, where every element, attribute, and piece of text is a node that can be accessed and manipulated by a scripting language like JavaScript.

Why JavaScript Needs the DOM

#	Reason	Explanation
1	Dynamic Content Manipulation	DOM allows JavaScript to change text, HTML, and attributes of a page <i>after</i> it has loaded — without needing to reload the page
2	Structured Access to Elements	The DOM organizes a document as a tree, letting JavaScript select any element via methods like <code>getElementById()</code> , <code>querySelector()</code>
3	Event Handling	DOM provides the mechanism for JavaScript to "listen" for user actions (clicks, key presses, form submissions) and respond accordingly
4	Dynamic Styling	JavaScript can modify CSS properties of elements in real time via the DOM (e.g., changing colors on hover/click)
5	Creating/Removing Elements	JavaScript can add new elements (<code>createElement()</code>) or delete existing ones (<code>removeChild()</code>) dynamically
6	Standardized Cross-Browser Interface	DOM is a W3C standard, so the same JavaScript DOM code behaves consistently across different browsers

Simple Worked Example

```
<p id="demo">Hello</p>
<button onclick="changeText()">Click Me</button>

<script>
// JavaScript accesses the DOM node with id="demo"
function changeText() {
    document.getElementById("demo").innerHTML = "Text Changed via DOM!";
}
</script>
```

In this example, JavaScript uses `document.getElementById("demo")` to locate a specific node in the DOM tree, then modifies its content directly — this is only possible because the browser exposes the page as a navigable, modifiable DOM tree.

Model answer: "JavaScript uses the DOM because the DOM represents the HTML page as a tree of objects that JavaScript can read, traverse, and modify in real time. Without the DOM, JavaScript would have no standardized way to locate specific page elements, respond to user events, or dynamically update content, styles, or structure after the page has loaded."

The `<font>` tag was historically used in HTML to control the appearance of text — its size, color, and typeface. **It is now deprecated (removed) in HTML5** in favor of CSS, but it remains an important topic for understanding HTML's evolution and is still commonly asked in sessional exams.

Attributes of the Font Tag

Attribute	Purpose	Example Values
<code>face</code>	Specifies the typeface/font family of the text	<code>face="Arial"</code> , <code>face="Times New Roman"</code>
<code>size</code>	Specifies the size of the text, on a relative scale of 1 (smallest) to 7 (largest); default is 3	<code>size="5"</code>
<code>color</code>	Specifies the color of the text, using a color name or hex code	<code>color="red"</code> , <code>color="#FF0000"</code>

Worked Example

```
<font face="Verdana" size="6" color="blue">
  Welcome to Web Technology!
</font>
```

Important note for exams: The `<font>` tag is **obsolete/deprecated in HTML5**. Modern web development uses CSS properties instead — `font-family` , `font-size` , and `color` — as shown below.

```
<p style="font-family:Verdana; font-size:28px; color:blue;">
  Welcome to Web Technology!
</p>
```

Exam tip: Always mention the deprecation note — many examiners specifically award marks for stating that `<font>` is obsolete and CSS is the modern replacement, since this shows deeper understanding beyond rote memorization.

The Internet evolved over several decades through a series of landmark developments. The five most commonly cited milestones are:

1969

ARPANET Launched — The Advanced Research Projects Agency Network (ARPANET), funded by the U.S. Department of Defense, became the first operational packet-switching network and the direct ancestor of the modern Internet, initially connecting four university computers.

1971

Email Invented — Ray Tomlinson developed the first system for sending messages between computers on a network, introducing the now-iconic "@" symbol to separate user names from host machine names.

1983

TCP/IP Adopted as Standard — ARPANET officially adopted the TCP/IP protocol suite, standardizing how data is transmitted across interconnected networks — this event is often marked as the "official birth" of the modern Internet.

1990

World Wide Web Invented — Tim Berners-Lee, working at CERN, invented the World Wide Web, along with HTML, HTTP, and the concept of URLs — creating the system of linked hypertext documents that made the Internet accessible to non-technical users.

1995 &
beyond

Commercialization & Mass Adoption — The mid-1990s saw the Internet open up to commercial use, the rise of web browsers like Netscape Navigator, search engines, e-commerce, and eventually broadband, mobile Internet, and social media — transforming it into the ubiquitous global infrastructure it is today.

Exam tip: Memorize the years alongside the milestone names — questions often specifically ask "in which year was X invented," so the dates themselves carry marks, not just the descriptions.

These are all **application-layer protocols** that define rules for specific types of communication over the Internet.

Protocol	Full Form	Function	Default Port
HTTP	HyperText Transfer Protocol	Governs how web browsers and servers exchange requests and responses to transfer web pages (HTML, images, etc.) across the Web	80
HTTPS	HyperText Transfer Protocol Secure	Same function as HTTP, but data is encrypted using SSL/TLS before transmission, protecting it from eavesdropping and tampering — essential for secure transactions (banking, login pages)	443
FTP	File Transfer Protocol	Used to upload and download files between a client and a server over a network, commonly used for website deployment and large file transfers	21
SMTP	Simple Mail Transfer Protocol	Used to send outgoing email from a client to a mail server, and between mail servers, across the Internet	25

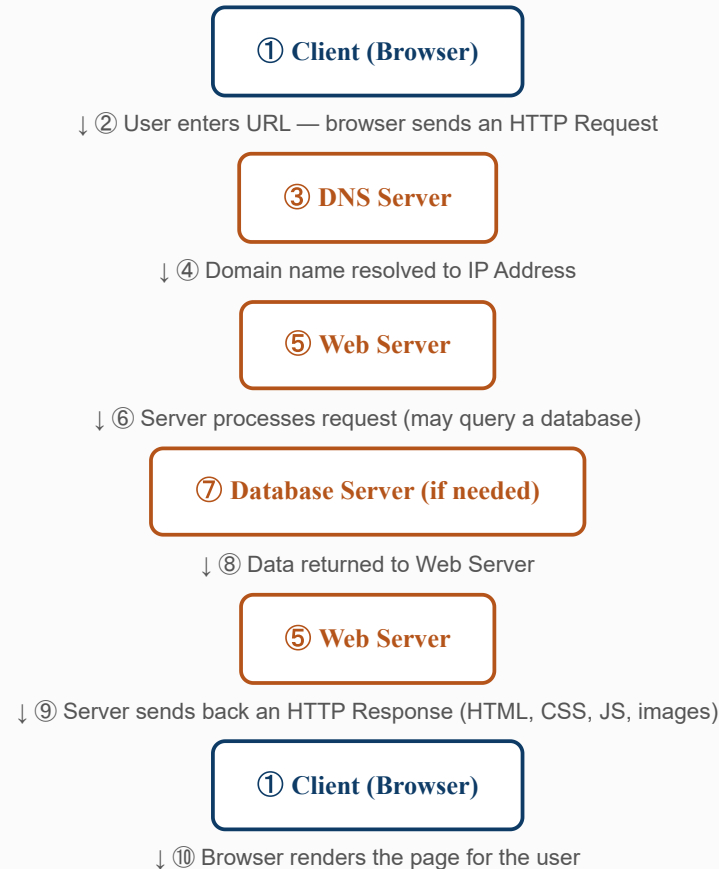
Detailed Function Breakdown

- **HTTP:** Follows a request-response model — the browser (client) sends an HTTP request for a resource, and the web server sends back an HTTP response containing the requested content (or an error code like 404).
- **HTTPS:** Adds a security layer (SSL/TLS) on top of HTTP, encrypting the data exchanged so that sensitive information (passwords, card numbers) cannot be intercepted in readable form by attackers — indicated by the padlock icon in browsers.
- **FTP:** Uses two separate connections — a control connection (for commands) and a data connection (for the actual file transfer) — allowing efficient transfer of large files, directory listing, and file management on a remote server.
- **SMTP:** Handles only the *sending* of email — retrieving/reading email typically uses separate protocols like POP3 or IMAP; SMTP relays messages from the sender's mail client to the recipient's mail server.

Model answer: "HTTP defines the rules for transferring web pages between browser and server; HTTPS is its secure, encrypted counterpart used for sensitive transactions; FTP handles uploading and downloading files between systems; and SMTP handles the transmission of outgoing email between mail servers. Together, these protocols enable the major categories of everyday Internet communication: browsing, secure transactions, file sharing, and email."

The **client-server architecture** is a computing model where tasks are distributed between two types of programs: a **client**, which requests services/resources, and a **server**, which provides/fulfills those requests. This is the foundational model behind virtually all web communication.

The Request-Response Cycle



Step-by-Step Explanation

Step	Description
1	User opens a browser and types a URL, or clicks a link
2	The browser (client) generates an HTTP/HTTPS request for that resource
3–4	The domain name is resolved into an IP address using DNS
5	The request reaches the correct web server
6–8	The server processes the request — if dynamic content is needed, it queries a database and retrieves the required data
9	The server packages the response (HTML page, JSON data, images, etc.) and sends it back to the client
10	The browser receives the response and renders the final web page for the user to view

Key Characteristics of Client-Server Architecture

- **Centralized resources:** The server holds the data/services; multiple clients can connect to it simultaneously
- **Request-response model:** Every interaction begins with a client request and ends with a server response
- **Statelessness (in HTTP):** By default, each HTTP request is independent — the server does not automatically remember previous requests from the same client (cookies/sessions are used to work around this)
- **Scalability:** Multiple servers (load balancers, clusters) can be used to handle a large number of simultaneous client requests

Exam tip: When "draw the diagram" is explicitly asked, always label each arrow with the step number/action (as shown above) — a diagram with unlabeled arrows loses marks even if the boxes are correct.

DTD (Document Type Definition) defines the **legal structure** of an XML (or HTML/SGML) document — it specifies which elements and attributes are allowed, their order, how many times they can appear, and their nesting rules. A DTD essentially acts as a "grammar" or "contract" that an XML document must follow to be considered **valid**.

Types of DTD

Type	Description
Internal DTD	Declared directly within the XML document itself, inside the <code><!DOCTYPE></code> declaration
External DTD	Declared in a separate <code>.dtd</code> file and referenced/linked from the XML document — allows reuse across multiple documents

Worked Example — Internal DTD

```
<?xml version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT note (to, from, heading, body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
<note>
  <to>Riya</to>
  <from>Asif</from>
  <heading>Reminder</heading>
  <body>Don't forget the exam!</body>
</note>
```

Worked Example — External DTD

note.dtd (separate file)

```
<!ELEMENT note (to, from, heading, body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

note.xml (referencing it)

```
<?xml version="1.0"?>
<!DOCTYPE note SYSTEM "note.dtd">
<note>
  <to>Riya</to>
  ...
</note>
```

Why Use a DTD?

- Ensures the XML document follows a consistent, agreed-upon structure
- Allows validation — checking whether an XML document is **well-formed** (correct syntax) *and* **valid** (conforms to the DTD's rules)
- Enables different systems/applications to reliably exchange XML data by agreeing on a common DTD "contract"

Model answer: "A DTD (Document Type Definition) defines the valid structure, elements, attributes, and their relationships for an XML document, acting as a grammar the document must conform to. It can be declared internally within the XML file itself, or externally in a separate .dtd file that can be shared and reused across multiple XML documents, enabling structural validation and consistent data exchange."

Below is a complete, well-organized external CSS stylesheet demonstrating common real-world styling techniques — layout, typography, navigation, buttons, and responsiveness.

```
/* ===== Reset & Base Styles ===== */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Segoe UI', Arial, sans-serif;
  line-height: 1.6;
  background-color: #f4f6f8;
  color: #333;
}

/* ===== Header & Navigation ===== */
header {
  background-color: #0d3b66;
  color: white;
  padding: 20px;
  text-align: center;
}

nav ul {
  list-style: none;
  display: flex;
  justify-content: center;
  background-color: #14508f;
}

nav ul li {
  margin: 0 15px;
}

nav ul li a {
  color: white;
  text-decoration: none;
  padding: 12px 8px;
  display: block;
}

nav ul li a:hover {
  background-color: #f4a261;
  color: #0d3b66;
}

/* ===== Main Content Layout ===== */
.container {
  max-width: 1000px;
  margin: 30px auto;
  padding: 20px;
  background: white;
  border-radius: 8px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
```

```

h1, h2 {
  color: #0d3b66;
  margin-bottom: 15px;
}

/* ===== Buttons ===== */
.btn {
  background-color: #f4a261;
  color: #0d3b66;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-weight: bold;
}

.btn:hover {
  background-color: #e08e3e;
}

/* ===== Footer ===== */
footer {
  text-align: center;
  padding: 15px;
  background-color: #0d3b66;
  color: white;
  margin-top: 30px;
}

/* ===== Responsive Design ===== */
@media (max-width: 600px) {
  nav ul {
    flex-direction: column;
  }
  .container {
    margin: 10px;
    padding: 10px;
  }
}

```

Matching HTML Structure (for reference)

```

<link rel="stylesheet" href="style.css">
<header>
  <h1>My Website</h1>
  <nav>
    <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">About</a></li>
    </ul>
  </nav>
</header>
<div class="container">
  <h2>Welcome</h2>
  <button class="btn">Click Me</button>
</div>
<footer>© 2026 My Website</footer>

```

Exam tip: A well-rounded "write CSS to design a web page" answer should touch on: selectors (element, class), the box model (padding/margin/border), a hover effect, and a media query for responsiveness — this example includes all four.

The `<form>` tag is a container used in HTML to collect user input and submit it to a server for processing. It groups together various input controls (text fields, buttons, checkboxes, etc.) into a single submittable unit.

Important Attributes of the Form Tag

Attribute	Purpose	Example
action	Specifies the URL/server-side script that will process the submitted form data	action="process.php"
method	Specifies the HTTP method used to send the data — GET (appends data to URL, visible) or POST (sends data in the request body, hidden)	method="post"
name	Assigns a name to the form, useful for referencing it via JavaScript	name="loginForm"
target	Specifies where to display the response after submission	target="_blank"
enctype	Specifies how form data should be encoded before sending (required as multipart/form-data for file uploads)	enctype="multipart/form-data"
autocomplete	Specifies whether the browser should autocomplete form fields based on prior entries	autocomplete="off"

GET vs POST — Quick Comparison

Feature	GET	POST
Data visibility	Appended to the URL (visible)	Sent in the request body (hidden)
Data length limit	Limited (URL length restrictions)	No practical limit
Security	Less secure — data exposed in browser history/URL	More secure for sensitive data
Best used for	Search queries, non-sensitive filters	Login forms, file uploads, sensitive data

Worked Example — Simple Login Form

```
<form action="login.php" method="post">

  <label for="uname">Username:</label>
  <input type="text" id="uname" name="username" required><br><br>

  <label for="pwd">Password:</label>
  <input type="password" id="pwd" name="password" required><br><br>

  <input type="submit" value="Login">

</form>
```

When the user clicks "Login," the browser collects the values of the `username` and `password` fields and sends them via an HTTP POST request to `login.php` for server-side processing (e.g., checking credentials against a database).

Model answer: "The `<form>` tag creates a container for collecting user input via elements like text fields, checkboxes, and buttons, and submits that data to a server-side script specified by the `action` attribute, using the HTTP method (GET or POST) specified by the `method` attribute. POST is preferred for sensitive or large data, while GET is suited to simple, shareable queries."